OpenMHP: An Open Protocol for Safe, Context-Efficient Agent Control of Laboratory and Industrial Instruments

Kushal Sinha
kushal.sinha@kortana.io
[TODO: affiliation, ORCID, any co-authors]

October 16, 2026

Abstract

Large language model agents have become competent at using software tools, largely through the Model Context Protocol (MCP), which standardises how an agent discovers and invokes a tool. Physical laboratory and industrial equipment resists the same treatment for three reasons: an operation may run for minutes or hours rather than returning immediately; the instrument emits telemetry, progress and safety events that the agent must observe while the work proceeds; and an incorrect request can destroy a sample or damage hardware, so correctness cannot be delegated to the model's judgement. We present OpenMHP (Open Model Hardware Protocol), an open protocol that extends the MCP interaction model to physical devices. OpenMHP contributes four design decisions. First, physical operations are modelled as *durable jobs* acquired under a *session lease*, where the lease doubles as a dead-man's switch: a device declaring a watchdog fails safe if the controlling process goes silent. Second, every request passes six *ordered safety gates* evaluated on the device side of the network, so the enforced envelope is independent of the model's context or intent. Third, each device is distributed as a *device package* structured like an Agent Skill, enabling progressive disclosure of operating knowledge; on a simulated 2,000-device laboratory, locating and operating one instrument costs 1,270 tokens against 1,224,382 tokens for naive descriptor loading, a reduction to 0.10%. Fourth, the protocol is explicitly a *bridging* layer: adapters expose OPC UA, ROS 2, SiLA 2, MADSci, PyLabRobot and MQTT devices without replacing existing control stacks. We describe the protocol, its trust boundary, a reference implementation in Python, and its conformance suite.

Keywords: laboratory automation; self-driving laboratories; AI agents; large language models; Model Context Protocol; robot safety; scientific instrumentation.

1 Introduction

Autonomous and semi-autonomous experimentation has moved from demonstration to practice. Systems such as Coscientist [26] and the A-Lab [17] have shown that a language model, given actuation and feedback, can plan and execute real experimental campaigns. In parallel, evolutionary and search-based agents such as AlphaEvolve [40] have demonstrated that agents can conduct genuine discovery when the feedback loop is fast and the object of study is symbolic. The corresponding loop in the physical sciences is far slower, and much of its latency is not scientific but

infrastructural: every instrument exposes a different programming interface, and connecting an agent to each one is bespoke work.

The software world addressed the analogous fragmentation with the Model Context Protocol (MCP) [1], which gives an agent one standard way to discover and call a tool, and a tool one standard way to describe itself. MCP's abstraction is a request that returns a response. That abstraction is a poor fit for physical work, in three specific ways.

Duration. A thermocycling protocol runs for two hours; a liquid-handling routine for twenty minutes; an incubation overnight. A tool call that blocks for the duration is not viable, and one that returns immediately discards the result.

Observation. While the instrument works it produces readings, progress, operator prompts and fault conditions. An agent that can only poll either wastes context or misses events. The natural shape is a push stream, which a request/response tool call cannot express.

Consequence. A malformed software call returns an error. A malformed physical call can overheat a reagent, crash a robot arm into a plate, or spoil an irreplaceable sample. Safety therefore cannot be a property of the prompt, the model, or the tool description, all of which are inside the agent's trust domain and all of which the model can, in principle, talk its way around.

OpenMHP is our response. It retains MCP's wire format, capability negotiation and host/client/server split, so that an OpenMHP deployment is usable from any MCP-speaking harness today, and adds exactly what the three gaps above require. This paper makes the following contributions:

1. A protocol design (Section 4) in which physical work is a durable job held under a session lease, with pushed notifications, six primitives, and a device descriptor that doubles as the instrument's manual.
2. A device-side safety model (Section 5) built on six ordered gates and an explicit trust boundary in which the host is trusted and the model is not, with fail-closed semantics for operations of unknown outcome.
3. A context-scaling mechanism (Section 6) combining descriptor detail tiers, a live-pinging directory, a constant tool surface and programmatic runs, measured at 0.10% of naive context cost on a 2,000-device laboratory.
4. An open reference implementation and conformance suite (Sections 8–9), including adapters for six existing control ecosystems.

2 Background and Related Work

2.1 Agent-facing protocols

MCP [1] standardises tool discovery and invocation for language-model applications over JSON-RPC 2.0 [44]. Its first year surfaced a scaling problem directly relevant to laboratories: loading every tool definition up front consumed roughly 55,000 tokens for five ordinary servers, and selection accuracy degraded as the list grew [3]. The reported mitigations — deferred tool search, programmatic tool calling, and usage examples embedded in definitions — inform Section 6. Agent Skills [4] established a complementary convention for packaging procedural knowledge that is loaded only when a task requires it; OpenMHP applies that convention to instruments.

	OpenMHP	MCP	SiLA 2	OPC UA	ROS 2	PyLabRobot	LabOP
Agent-facing natural-language description	✓	✓	–	–	–	–	–
Long-running work as a durable job	✓	–	✓	–	✓	–	–
Server-pushed observation	✓	–	✓	✓	✓	–	–
Device-side enforced safety envelope	✓	–	–	–	–	–	–
Explicit human-approval channel	✓	✓	–	–	–	–	–
Exclusive control with expiry (lease)	✓	–	✓	–	–	–	–
Liveness-linked fail-safe (watchdog)	✓	–	–	–	–	–	–
Bounded agent context at fleet scale	✓	✓	–	–	–	–	–
Procedure description	–	–	–	–	–	✓	✓
Vendor-neutral actuation	✓	–	✓	✓	✓	✓	–

Table 1: Where OpenMHP sits. The comparison is deliberately uncharitable to OpenMHP on the bottom two rows: it describes no procedures and performs no actuation of its own, delegating both. Its contribution is the combination in the middle of the table. A dash means the concern is out of scope for that layer rather than done badly; SiLA 2 and OPC UA are mature machine-to-machine standards that were not designed to be read by a language model, and MCP was not designed for work that outlives a call or for enforcement the caller cannot reach.

2.2 Laboratory and industrial control standards

SiLA 2 [32] provides a gRPC-based standard for laboratory instrument interfaces with strong typing and feature definitions. OPC UA [33] is the dominant information model in industrial automation. ROS 2 [30] supplies the middleware for most research robotics. MADSci [34, 35] and PyLabRobot [31] provide orchestration and hardware-agnostic liquid handling respectively. These layers solve machine-to-machine integration well, and OpenMHP does not attempt to replace them. What none provides is an *agent-facing* surface: a natural-language account of what an instrument is for, when not to use it, and which operations require a human, together with a context budget suitable for a model. Table 1 summarises the division.

2.3 Protocol and procedure description languages

Autoprotocol and LabOP [36] describe experimental procedures in machine-readable form, addressing *what* to run. OpenMHP is complementary and addresses *how an agent safely drives a specific instrument*, including the run-time envelope that a procedure description cannot enforce.

2.4 Self-driving laboratories

Closed-loop experimentation is now an established field, surveyed by Abolhasani and Kumacheva [22] and, at length, by Tom et al. [23]. ChemOS [18] and its successor ChemOS 2.0 [19] orchestrate autonomous campaigns; Ada [20] demonstrated a self-driving laboratory for thin-film materials, with design guidance later distilled by MacLeod et al. [21]; Burger et al. [16] showed a mobile robot operating existing instruments rather than a purpose-built rig; and the A-Lab [17] ran an autonomous inorganic synthesis campaign. Snapp and Brown [25] discuss the practical obstacles to building such systems.

These systems are typically one facility, one software stack and one orchestrator, with integration performed once by the facility's own engineers. OpenMHP targets the heterogeneous case: an ordinary laboratory with instruments from many vendors, several control stacks, and no single orchestrator, where the agent is supplied by the user rather than the facility. Burger et al. is the closest in spirit, in that the robot adapts to instruments rather than the reverse.

2.5 Language-model agents for science

Coscientist [26] and ChemCrow [27] demonstrated that a language model equipped with tools can plan and execute chemical work, and more recent systems extend the loop to writing up results [28]; the area is surveyed by Wei et al. [29]. This literature concentrates on the reasoning layer and generally assumes an actuation interface exists. OpenMHP is a proposal for what that interface should be, and in particular for placing the safety envelope outside the agent's reach, which becomes more important as the reasoning layer becomes more autonomous.

2.6 Tool retrieval and context management

The problem of too many tools is well studied. Toolformer [7] trains a model to decide when to call an API; Gorilla [5] and ToolLLM [6] address retrieval over large API collections; and retrieval-augmented tool selection has been shown to mitigate prompt bloat directly [8, 9]. Evaluation of tool use at scale is itself an active area [10]. OpenMHP's contribution here is not a better ranker but the observation, supported in Section 9, that a protocol which exposes structured filters alongside free text keeps retrieval stable where ranking alone does not. Security analyses of MCP deployments [2] are also relevant to the trust boundary of Section 3.

2.7 Relation to workload managers and workflow engines

It is tempting to describe OpenMHP as a scheduler for laboratory instruments, and the analogy should be stated carefully. Workload managers such as SLURM [37] queue work, allocate resources among competing users and enforce priority; workflow engines such as Nextflow [38] and Snakemake [39] express a dependency graph and execute it reproducibly. OpenMHP does none of these: it has no cross-device queue, no scheduling policy and no notion of competing users beyond lease exclusivity. The accurate correspondence is to the per-node component of such a system rather than to its controller. An OpenMHP server supervises work on one instrument, enforces its limits and terminates it when it misbehaves, in the way a node daemon does for a compute node; the decision of what to run where has no equivalent and would naturally be built above the protocol. Two narrower correspondences do hold, and are deliberate: the session lease resembles exclusive resource allocation with a timeout, and the durable job handle resembles a batch job identifier.

2.8 Safety engineering

OpenMHP borrows its vocabulary from functional safety [41] and robot safety practice [42, 43] without claiming conformance to either; Section 5 states the boundary. The lease mechanism is the distributed-systems construct of Gray and Cheriton [11], applied to physical actuation: a time-bounded grant whose expiry is itself informative. The difference is the consequence of expiry. In a distributed file cache a lapsed lease invalidates cached data; on an instrument it must drive the hardware to a safe state, which is why the watchdog of Section 4 is coupled to lease liveness rather than to request traffic.

3 Design Goals and Trust Model

OpenMHP is specified against five rules, in priority order.

1. **Safety is enforced by the driver, never by the agent.** Limits, interlocks and approval levels live in the descriptor and are checked on the device side of the wire.
2. **One descriptor is the whole manual.** Tacit operating knowledge is carried in natural-language `notes` fields alongside machine-readable types and limits.
3. **Six primitives, no more.** A device that speaks `describe`/`signals`/`settings`/`actions`/`safety`/`methods` can be operated by any OpenMHP host.
4. **Bridge, do not replace.** Existing control layers become OpenMHP devices through thin adapters.
5. **The agent's context is a protected resource.** A laboratory with two thousand devices costs approximately what a laboratory with two costs.

Trust boundary. The central assumption is that *the host is trusted and the model is not.* The hazard being avoided is a confused deputy [13]: a privileged component acting on authority it holds but that the requester does not. A model asserting `approved: true` about its own request is exactly that, which is why the flag is treated as the host's statement and is stripped from anything the model supplies. The host is the agent runtime; it holds clients, presents confirmation prompts to people, and is assumed to execute faithfully. The model it hosts is treated as an untrusted source of requests. Concretely, the flag `approved: true` is the host's assertion that a person confirmed one exact request; a reference bridge strips and ignores any `approved` value originating from the model. Package code, recipes and orchestration scripts are executed by the host and therefore sit inside the trusted domain: a deployment in which the model can edit a device package or run unsandboxed scripts has moved the enforcement boundary, and we state this explicitly rather than claiming unconditional guarantees (Section 10).

4 Protocol Design

4.1 Architecture

An OpenMHP deployment comprises a *host* (the agent runtime), one *client* per device, and one *server* per device. A server owns exactly one logical device, serves its descriptor, enforces the safety envelope, translates primitives into vendor commands and runs jobs. A server may equally be an *adapter* wrapping another control layer; from the host's perspective the two are indistinguishable. Messages are JSON-RPC 2.0 over either stdio or HTTP with Server-Sent Events.

4.2 Primitives

The protocol defines six primitives: `device/describe` (identity and capability, see Section 6); `signals/read` (sensed values); `settings/write` (setpoints, limit-checked); `actions/invoke` (long-running work, returning a job); `safety/*` (`estop`, `reset`); and `methods/*` (named, versioned parameter sets filed per project or compound, for instruments such as chromatographs that operate by named method rather than by free parameters). Session management (`session/acquire`, `renew`, `release`) and job control (`jobs/status`, `pause`, `resume`, `cancel`) support these.

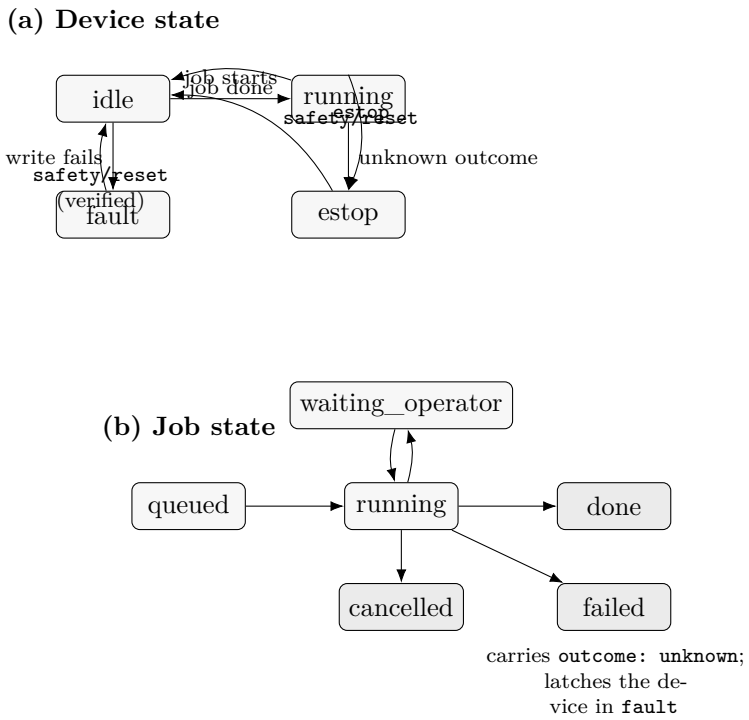


Figure 1: (a) Device state and (b) job state. The two are coupled at one point, drawn as the `failed` annotation: a job that fails with an unknown outcome latches the device into `fault`, from which there is no automatic recovery. Recovery requires `safety/reset`, which verifies the instrument before returning it to `idle`. `waiting_operator` allows an instrument whose steps are performed by a person to participate in the same job model as an automated one.

4.3 Jobs

`actions/invoke` returns immediately with a job identifier rather than a result. Jobs progress through `queued` → `running` → `done` | `failed` | `cancelled`, with an additional `waiting_operator` state for instruments whose steps are performed by a person. The agent may disconnect and reconnect without losing the state of the physical process. A failed job carries `outcome: "unknown"` and places the device in `fault` until a verified `safety/reset`, on the principle that a partially completed physical action must not be assumed benign.

4.4 Leases and the watchdog

`session/acquire{ttl}` grants the calling session exclusive write and invoke rights until the TTL lapses without renewal or the session releases them. Every client has its own session identifier, so independent clients never share a lease. Reads, `ping`, `describe`, `jobs/status`, `jobs/pause`, `jobs/cancel` and `safety/estop` are deliberately never blocked by a lease, so that observation and stopping remain always available.

The lease additionally serves as a dead-man's switch. If a descriptor declares `safety.watchdog_s`, the watchdog is armed whenever a session holds the lease; any request from the holder renews it. Should the holder fall silent for longer than the declared window, or should its lease expire without release, the server must fail safe: enter `estop`, cancel active jobs, run the stop hook and drop the lease. Recovery requires a verified reset. This converts the liveness of the

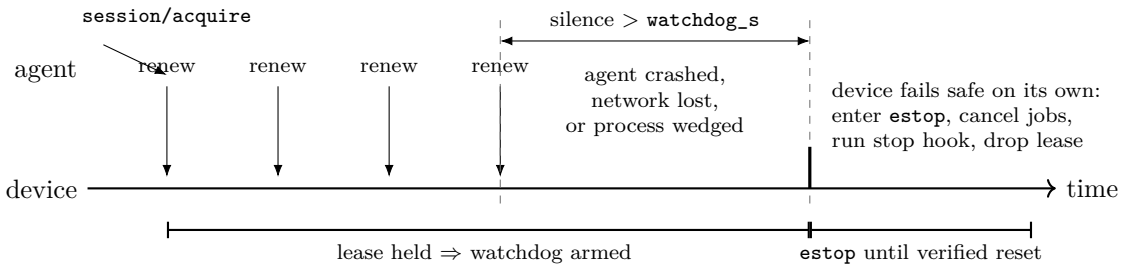


Figure 2: The lease as a dead-man's switch. The watchdog is armed for exactly as long as a session holds the lease, and any request from the holder renews it. Nothing needs to detect that the agent has failed: the absence of renewal is itself the signal. After the declared window the server acts alone, without the agent and without the network, and recovery requires a verified `safety/reset`. This converts liveness of the controlling process into a safety property, and is the clearest respect in which driving an instrument differs from calling a software tool.

controlling process into a safety property, which we regard as the single most important difference between driving an instrument and calling a software tool.

4.5 Notifications

Servers push JSON-RPC notifications for signal updates, job start, progress, completion, pause/resume/cancel, e-stop, fault, operator instructions, and method changes. A supervising host consumes these without being asked; the reference bridge subscribes to every device it opens, records events against the owning run, and writes notable ones to an append-only run log. No part of this path polls the instrument.

4.6 The device package

A device is distributed as a folder, deliberately shaped like an Agent Skill:

```
devices/hotplate-01/
  DEVICE.md        # YAML frontmatter (card) + operating instructions in prose
  descriptor.yaml  # signals, settings with limits, actions, interlocks, safety
  driver.py        # a Driver subclass, a BoundDriver, or an adapter
  sim.py           # optional simulated twin
  references/      # SOPs, manual excerpts
  scripts/         # tested orchestration scripts
  methods/         # optional saved methods per project
```

The separation matters: `DEVICE.md` carries what a new colleague would be told, in the owner's own words, while `descriptor.yaml` carries what a machine must enforce. The same artefact therefore serves both the model's understanding and the driver's gate checks, and the two cannot drift apart silently.

4.7 Composing several instruments

A real experiment rarely involves one instrument. An electrochemistry run may drive a pressure controller, a selection valve, a potentiostat, a Raman spectrometer and a gas chromatograph together. OpenMHP composes these horizontally rather than hierarchically (Figure 3).

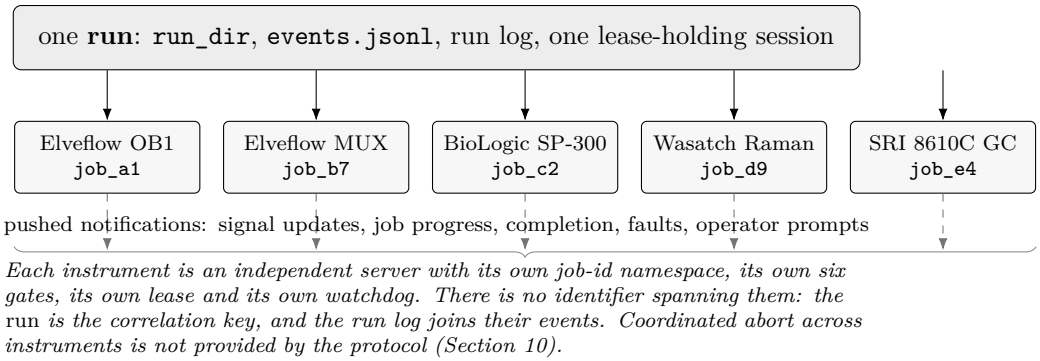


Figure 3: A five-instrument electrochemistry experiment. The composition is horizontal rather than hierarchical: no instrument knows about any other, and the orchestrating run holds one lease per device from a single session. This keeps each device's safety envelope independent and local, at the cost of leaving cross-device atomicity to the run script.

All devices reached through one client share that client's session identifier, and a lease is held per *(device, session)* pair. One agent may therefore hold leases on any number of instruments simultaneously, while each lease remains exclusive against every other session. Nothing in the protocol limits the size of that set, which matters because the alternative, a single global lock over the laboratory, would serialise experiments that have no physical interaction.

What the protocol deliberately does not provide is a shared identifier across instruments. Each server issues job identifiers in its own namespace, so there is no distributed transaction and no cross-device job. Correlation happens one level up: the orchestrating run has an identifier, a directory, an event file and a log, and every call made under it is tagged, so the five instruments above are joined by the run rather than by the job. We regard this as the right default, because it keeps each device's safety envelope local and independent, but it has consequences that Section 10 states plainly.

5 Safety Model

5.1 Six ordered gates

On every `settings/write` and `actions/invoke`, the server evaluates, in this fixed order:

1. **State** — `estop` yields `EStopActive`; `fault` yields `DeviceFault`.
2. **Approval** — `forbid` yields `Forbidden`; `confirm` without `approved: true` yields `ApprovalRequired`.
3. **Interlocks** — each named signal is read live and must be exactly `true`; false, unreadable or stale yields `InterlockOpen`.
4. **Values** — type, then declared limits; for actions, declared, required and bounded parameters, then the driver's whole-request `validate_params`.
5. **Lease** — a lease held by another session, or a watchdog device operated without one, yields `NotLeased`.
6. **Busy** — a non-concurrent action while a job runs yields `DeviceBusy`.

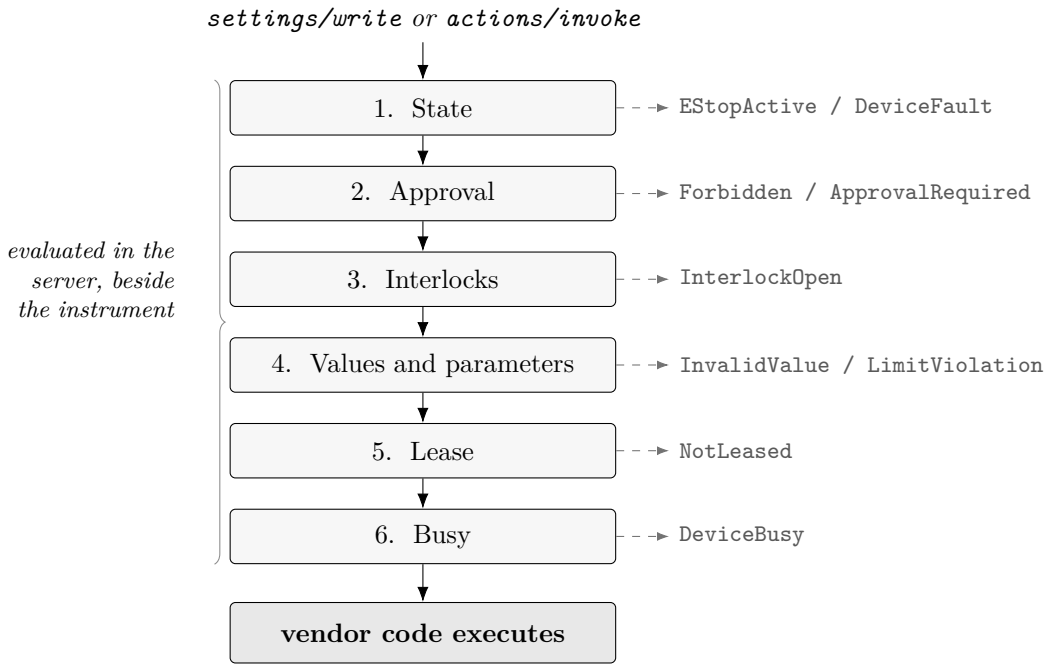


Figure 4: The six gates, evaluated in this fixed order on every write and invocation. Each gate has exactly one refusal, shown on the right. Vendor code runs only when all six pass. The order is normative because it determines which refusal an agent observes first, and therefore what it can learn from being refused. Every gate is evaluated on the device side of the network, so the enforced envelope does not depend on the model's context, prompt, or intent.

Vendor code executes only after all six pass. The order is normative because it determines which refusal an agent sees first, and therefore what it learns from a refusal.

5.2 Human approval and elicitation

Approval level `confirm` is the junction between autonomy and oversight. The sequence is: the model requests the action; the host receives `ApprovalRequired`; the host presents the device, the action, its exact parameters and the descriptor's notes to a person; and only on an explicit affirmative does the host resend the identical request with `approved: true`. The reference bridge implements this through MCP elicitation. Where a harness offers no elicitation channel, the bridge has no trusted path to a person, and confirm-gated operations are reported unavailable rather than being silently permitted.

5.3 Fail-closed semantics and defence in depth

The device is designed to fail into a known state rather than an arbitrary one, in the sense of fail-stop processors [12] and the dependability taxonomy of Avižienis et al. [15]. Any exception inside vendor code during a job, and any write failure occurring after a command may already have reached the instrument, moves the device to `fault` with an unknown outcome and runs the driver's safe-state hook. There is no automatic recovery. Separately, limits on scalar setpoints are insufficient when a dangerous value is nested inside a structured parameter — for instance a 200 °C step inside an otherwise valid protocol — so the driver's `validate_params` inspects the whole

request, and must do so for dry runs as well, ensuring no step executes before the entire program is known valid.

5.4 Scope

OpenMHP is not a functional-safety system in the IEC 61508 [41] sense. Hard-wired emergency stops, light curtains and PLC safety logic remain the primary protective layer. OpenMHP prevents an agent from *asking* for something unsafe and gives it a standard way to stop everything when it observes a problem. We state this prominently because conflating the two would be the most consequential misreading of this work.

6 Context Scaling

A laboratory has more devices than a workspace has tools, and a device descriptor is longer than a tool schema. Four mechanisms keep the agent's context approximately independent of laboratory size.

Detail tiers. `device/describe` accepts a `detail` parameter with three tiers: `card` (roughly 40 tokens: identity, location, live state and the *names* of capabilities), `summary` (under 1,000 tokens: the card plus operating instructions plus every capability with type, unit, limits, approval level and interlocks), and `full`. A `select` parameter fetches complete specifications for named items only. The intended path is card → summary → select the two items that will actually be used.

Directory. A directory is a server whose role is finding devices rather than being one. It indexes cards only, ranking with BM25 [45] over identity, class, make, model, location, tags, notes and capability names, so a query such as *"heat a 96-well plate to 95 °C for PCR in bay 12"* resolves without the agent knowing any identifiers. Crucially, *state is never served from the index*: at query time the directory pings top candidates in parallel with a short timeout and fills in live state before applying any state filter, so a stale index cannot cause an agent to address an unreachable instrument.

Constant tool surface. A bridged host sees eleven tools regardless of device count, and never per-device tools. Bridges must not enumerate devices into the tool list at startup.

Programmatic runs. An orchestration script executes against the laboratory and returns only what it prints, capped. A script that takes five hundred readings and reports a mean places one line in the model's context rather than five hundred numbers. A plan mode rehearses a script with reads real and every mutation a dry run through the same gates, under virtual time and a step budget, producing the ordered steps, those requiring confirmation, those that are long-running, and the first step the driver would refuse.

7 Bridging Existing Ecosystems

All adapters are built on one mechanism. A *binding* pairs an OpenMHP name with a callable into the foreign layer, and `BoundDriver` turns three lists of bindings into a complete driver that inherits every gate, tier, job and notification:

```
from openmhp.adapters import BoundDriver, Signal, Setting, Action
dev = BoundDriver(
    device={"id": "hotplate-01", "class": "hotplate", "notes": "Fume␣hood␣2."},
    signals=[Signal("plate_temperature", read=lambda: plc.read(0x10), unit="degC")],
    settings=[Setting("target_temperature", write=lambda v: plc.write(0x20, v),
                      limits={"min": 20, "max": 300})],
    actions=[Action("shutdown", run=lambda job, p: plc.write(0x21, 0),
                    approval="confirm")],
    estop=lambda: plc.write(0x21, 0))
```

The reference implementation ships adapters for OPC UA, ROS 2, SiLA 2, MADSci, PyLabRobot and MQTT on this one mechanism. The practical consequence is that the safety envelope is added *above* a control layer that may itself provide none, without modifying it.

8 Reference Implementation

`openmhp` is a Python implementation of the specification, comprising approximately 7,300 lines across the driver base class, package loader, directory, stdio and HTTP+SSE transports, client SDK, command-line interface, MCP bridge, the six adapters, three bundled Agent Skills, simulated devices and the scale demonstration. PyYAML is the only runtime dependency of the core. Writing a new driver requires a descriptor and three hooks (`on_read`, `on_write`, `on_invoke`), with optional `validate_params` and `on_fault`.

Four conformance levels are defined: **Core** (describe with all tiers, read, limited write, error codes), **Actions** (jobs, interlocks, approval gates, methods), **Safe** (non-blocking e-stop, verified reset, watchdog, fail-closed state machine, per-session leases) and **Directory** (the server role). Devices carrying stored energy or capable of motion should not be exposed below Safe.

9 Evaluation

9.1 Context cost at scale

The reference `examples/scale_demo.py` constructs 2,000 simulated devices across 8 classes and 40 bays, and measures the tokens entering an agent's context for a representative task: find a suitable instrument, understand it, and operate it.

Strategy	**Tokens in agent context**
Every descriptor loaded up front	1,224,382
Find (5 cards) + summary of chosen device + full spec of the 2 items used	1,270

Table 2: Context cost on a simulated 2,000-device laboratory. The progressive path costs 0.10% of naive loading, with retrieval well under one millisecond. The device is then operated through exactly the same primitives as in a two-device laboratory.

Figure 5 extends this single measurement into a curve from 10 to 10,000 devices. Naive loading grows linearly, exceeding six million tokens at the largest size, while the progressive path stays flat at roughly 1,270 tokens.

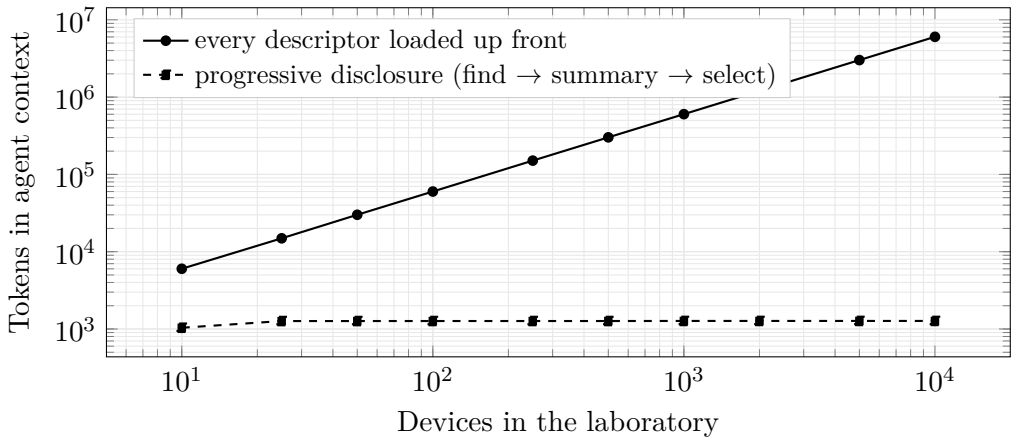


Figure 5: Context cost against laboratory size. Naive loading grows linearly with device count, exceeding six million tokens at 10,000 devices. The progressive path is effectively flat at roughly 1,270 tokens, because the agent reads five cards, one summary and two selected specifications regardless of how many instruments exist. Token counts use the proxy described in Section 9.

9.2 Retrieval under growth

A flat context cost is only useful if the right instrument is still found. Figure 6 measures this with 200 trials at each size, where a query states a task and a bay and a trial succeeds when the first result is an instrument of the class that performs that task, in the requested bay. Free-text ranking alone degrades from 86% to 59% as the laboratory grows, reproducing the selection-accuracy decay reported for large tool sets [3]. Adding the class filter recovers most of it, and class together with location holds accuracy at 99.5–100% across three orders of magnitude, at a median search cost that remains in single-digit milliseconds. The stability is therefore a property of the protocol's structured query surface, not of the ranking function, which is an argument for specifying such filters rather than leaving retrieval to free text.

9.3 Integration effort

Figure 7 counts driver source lines for each package in the device catalogue. The two no-code routes cost one and two lines, because the behaviour is declared in `descriptor.yaml` rather than written in Python; wrapping an existing ecosystem costs eleven; only proprietary vendor SDKs require 98 to 224 lines. In every case the gates, tiers, jobs and notifications are inherited rather than reimplemented.

9.4 Conformance and behavioural testing

Conformance in OpenMHP is behavioural rather than structural. The reference suite comprises 53 tests across approximately 1,800 lines, exercising typed limits and limit violations, lease ownership and contention, watchdog expiry under induced silence, rejected and forged approvals, interlock enforcement, blocked I/O, fail-closed fault latching on backend aborts, plan-mode isolation from real actuation, and the method lifecycle. Adapters are tested against injected fakes of each vendor client library, verifying that the gate behaviour is inherited correctly through `BoundDriver` rather than reimplemented per ecosystem.

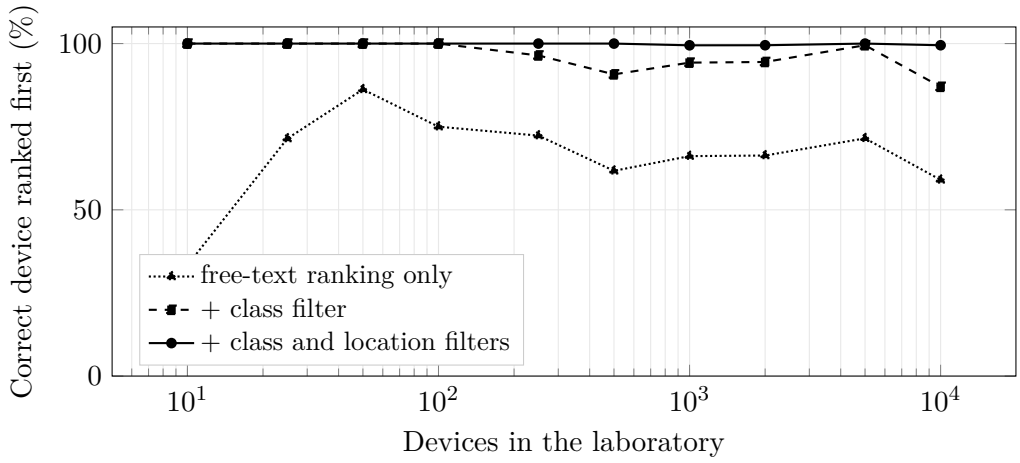


Figure 6: Retrieval under growth, 200 trials per point. A query states a task and a bay; a trial succeeds when the first result is an instrument of the class that performs the task, in the requested bay. Free-text ranking alone degrades from 86% to 59% as the laboratory grows, reproducing the selection-accuracy decay reported for large tool sets [3]. The structured filters that `directory/search` already accepts, and that the bridge issues, hold accuracy at 99.5–100% across three orders of magnitude. The stability comes from the protocol's structure rather than from the ranker.

9.5 Deployment on physical instruments

[TODO: This subsection must be written from your real hardware runs before submission; do not submit the paper without it. Reviewers will expect, at minimum: (i) which instruments, by make and model, and which conformance level each was exposed at; (ii) which integration route each used — native driver, serial ASCII, or which adapter; (iii) how long each ran and how many jobs completed; (iv) at least one instance of a gate firing against a real unsafe request, with the refusal; (v) at least one watchdog or e-stop event observed on real hardware, since that is the central safety claim of Section 5; (vi) any failure, surprise or workaround, which reviewers weight heavily in a systems paper; and (vii) whether a domain scientist, rather than the authors, operated it. If a controlled comparison is possible — time or lines of integration code for the same instrument with and without OpenMHP — it would substantially strengthen the paper.]

9.6 Assessment against published procurement criteria

Self-assessment by a protocol's authors is weak evidence. Gottstein et al. [24] recently published twenty-one RFC-2119 criteria defining an autonomous-ready control interface, with per-criterion tests and procurement weights, drawn up across the self-driving-laboratory community including the SiLA and OPC UA LADS standards bodies. The criteria were not written with OpenMHP in view, which makes them a more useful yardstick than any we could propose. Table 3 scores OpenMHP against all twenty-one.

The weighted result is 39.0 of 46 for a standalone instrument, and 41.0 of 50 including the two cell-integration criteria that apply to a protocol. Three findings matter more than the total.

First, the single highest-scoring item is C15, control-loss observability, weighted 3 and passed

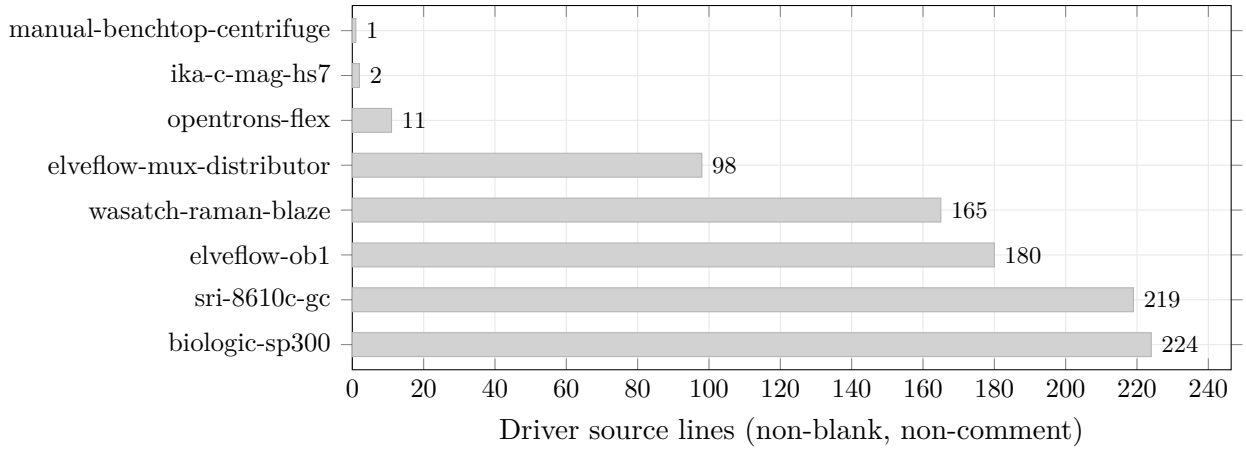


Figure 7: Integration effort per instrument, measured on the packages in `supported-devices/`. The two no-code routes cost one and two lines: a human-operated centrifuge is driven entirely by operator prompts, and the IKA hotplate by declared serial commands, with the behaviour expressed in `descriptor.yaml` rather than in Python. Wrapping an existing ecosystem costs eleven lines through the PyLabRobot adapter. Only instruments reached through a proprietary vendor SDK require a substantial driver, between 98 and 224 lines. The safety envelope, detail tiers, jobs and notifications are inherited in every case.

outright. This is the criterion that asks what happens when the controlling client dies while hardware is active, and it is the one the lease-armed watchdog of Section 4 exists to answer. That the independently derived criterion and the mechanism coincide is the strongest external evidence we have that the design addresses a real requirement rather than a self-selected one.

Second, all five Critical criteria now pass, and three of them did not when we first scored. Each was closed by a specification change the exercise identified rather than by reinterpreting the criterion: C3 gained a device-anchored timestamp on the error envelope; C2 gained declared resolution, nullability and defaults, the last reported back on every response and on the job so a substituted value is never applied in silence; and C5 gained an optional result contract. We regard this as the scorecard's main value to us. Three gaps that we had not identified in two years of designing the protocol were found in an afternoon by scoring it against criteria somebody else wrote. Declaring a result contract is optional, which was the condition for adding one at all: instrument outputs vary so widely that a mandatory schema would be wrong more often than useful, while an optional one lets a chromatograph declare its peak table and a camera declare nothing. Each remaining gap is a small, backward-compatible change, and together they would move the Critical tier from 13.5 of 18 to 18 of 18. We state them here rather than defer them because the same paper observes that protocol compliance does not establish laboratory interoperability, and these three are exactly where that gap opens.

Third, the only outright Fail is C12, instrument self-state: the protocol defines no place for calibration status, qualification state, usage counters or out-of-spec indications. A driver may expose them as ordinary signals, but because nothing names them, no two devices will agree, which reproduces at the semantic layer the fragmentation the protocol removes at the transport layer. This is the clearest argument for aligning the device-class vocabulary of Section 4 with an established laboratory information model rather than continuing to treat classes as hints.

#	Criterion	Wt	Score	Evidence and reservation
Critical				
C1	Reachable control	3	Pass	stdio and HTTP+SSE endpoints; `/mhp.json` needs no handshake; a reconnecting client resolves a command's fate by job id, not by inference.
C2	Typing, units	3	**Pass**	Signals, settings and action parameters declare `type`, `unit`, `limits`, `resolution`, `nullable` and `default`, nested to any depth, with UCUM as the declared vocabulary. An off-resolution value is refused naming the nearest settable one rather than silently rounded, and an applied default is reported as `defaulted` on the response and on the job. Reading "typed outputs" as action results rather than signals would tie this to the same gap as C5.
C3	Structured errors	3	**Pass**	All four sub-tests: 13 declared codes, a message, a populated structured context (`LimitViolation` carries the violated bound, `InvalidValue` the expected type, `NotLeased` the holder and reason), and a device-anchored `ts` on every error envelope.
C4	Open API access	6	Pass	Apache 2.0, no royalties, integrations publishable and reusable, documentation free. Caveat: a device behind a proprietary SDK inherits that vendor's terms; the protocol cannot launder them.
C5	Result retrieval	3	**Pass**	Headless and execution-linked: the job returns the result with its id, state, outcome, timings and owning method. Actions may now declare `returns`, a typed result contract in the same vocabulary as `params`; a result that breaks it is reported as `resultMismatch` rather than raised, since the physical work has already happened. Declaring it is optional, so an instrument with no stable output shape is not forced to invent one.
Important				
C6	Runtime self-description	2	Pass	`device/describe` with three tiers and `select`; a generic client enumerates the surface with no prior knowledge.
C7	Observable state	2	Pass	`jobs/status` never blocks on completion; progress, signals, safety and operator events are pushed.
C8	Reference client	2	Pass	Python SDK, CLI, MCP bridge, worked examples and tested recipes.
C9	Concurrent access	2	Partial	Atomic `session/acquire` with TTL, visible holder, renewal and reconnect. No role-gated administrative clear.
C10	Device/readiness state	2	Pass	Named states with machine-readable recovery through verified `safety/reset`.
C11	Result durability	2	Partial	Run records persist on disk with stable ids; the device job table is in memory, so a server restart loses job-level results.
C12	Instrument self-state	2	**Fail**	The protocol defines no place for calibration state, qualification, usage counters or out-of-spec flags. A driver may expose them as signals, but nothing names or requires them, so no two devices agree.
C13	Versioning discipline	2	Partial	`PROTOCOL_VERSION` is negotiated at `initialize` and echoed in `describe`. No deprecation policy and no support commitment; pre-1.0 states that breaking changes are expected.

10 Limitations and Future Work

We state the boundaries of the current draft explicitly.

- **Not functional safety.** OpenMHP does not replace IEC 61508-class protective systems (Section 5).
- **Enforcement depends on the trust boundary holding.** A model able to edit a device package, or to run arbitrary scripts in an unsandboxed host, can alter what is enforced. Deployments should separate package authoring from operation.
- **No coordinated abort across instruments.** A run holding leases on several devices releases them on exit, but the protocol does not cancel sibling jobs or stop sibling devices when one faults. In a five-instrument electrochemistry run, a potentiostat fault does not by itself stop the pumps. Today this is the orchestrating script's responsibility; a `transaction/` namespace is the natural home for it.
- **No common time base across instruments.** Events carry per-device timestamps from independent clocks, and the protocol defines no happens-before relation across servers [14]. Operando work that must align a spectrum with an electrochemical trace currently depends on the run script to reconcile timing.
- **No lock ordering.** A session may hold leases on any number of devices, but there is no atomic multi-acquire and no prescribed acquisition order, so two agents requesting overlapping sets in different orders can make no progress until leases lapse.
- **Non-idempotent operations.** A request may reach a device even when its response is lost, and servers do not deduplicate. Clients must not retry writes or invocations automatically; they should reconcile by reading state.
- **Unspecified in this draft.** In-band authentication, and authentication for the device HTTP transport; idempotency tokens; multi-device transactions; descriptor signing; a hosted registry; sandboxed script execution; streaming of bulk data; and a machine-readable description of hardware watchdogs and device-side safe states.
- **Evaluation breadth.** **[TODO: state honestly how many instruments, of what diversity, and in how many independent laboratories; note explicitly if all deployments are by the authors.]**

11 Conclusion

Agents are becoming capable enough that the limiting factor in physical science is increasingly the interface to the instrument rather than the reasoning about the experiment. OpenMHP proposes that this interface should be open, should model physical work as durable leased jobs with pushed observation, should enforce its safety envelope on the device side of the network where the model cannot reach it, and should package instrument knowledge for progressive disclosure so that laboratory size does not consume the agent's context. The specification, reference implementation and conformance suite are available under Apache 2.0.

Availability

Specification and reference implementation: https://github.com/kushalsinha/openmhp. Documentation: https://openmhp.com. Released under the Apache License 2.0. **[TODO: mint an archival DOI (e.g. via Zenodo) for the exact release this paper describes, and cite it here; journals increasingly require it and it is what others will cite.]**

Acknowledgements

[TODO: acknowledgements; funding statement; and a disclosure of AI assistance in the preparation of the software and/or manuscript, which arXiv and most journals now require.]

References

[1] Anthropic. *Model Context Protocol*: specification and documentation. Introduced 25 November 2024. https://modelcontextprotocol.io (accessed October 16, 2026). There is no peer-reviewed publication describing MCP; the specification is the citable artefact.

[2] X. Hou, Y. Zhao, S. Wang, H. Wang. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278, 2025.

[3] Anthropic. *Advanced tool use*. Engineering blog. **[TODO: verify exact title, author and date before submission]**

[4] Anthropic. *Agent Skills*. **[TODO: verify citation form]**

[5] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems 37* (NeurIPS 2024). DOI: 10.52202/079017-4020.

[6] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations* (ICLR 2024).

[7] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36* (NeurIPS 2023), pp. 68539–68551.

[8] T. Gan, Q. Sun. RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation. arXiv:2505.03275, 2025.

[9] E. Lumer, V. K. Subbiah, J. A. Burke, P. Honaganahalli Basavaraju, A. Huber. Toolshed: Scale Tool-Equipped Agents with Advanced RAG-Tool Fusion and Tool Knowledge Bases. arXiv:2410.14594, 2024.

[10] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, J. E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning* (ICML 2025), PMLR 267, pp. 48371–48392.

[11] C. Gray, D. R. Cheriton. Leases: an efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles* (SOSP '89), pp. 202–210, 1989. DOI: 10.1145/74850.74870.

[12] R. D. Schlichting, F. B. Schneider. Fail-stop processors: an approach to designing fault-tolerant computing systems. *ACM Transactions on Computer Systems*, 1(3):222–238, 1983. DOI: 10.1145/357369.357371.

[13] N. Hardy. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. DOI: 10.1145/54289.871709.

[14] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. DOI: 10.1145/359545.359563.

[15] A. Avižienis, J.-C. Laprie, B. Randell, C. E. Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004. DOI: 10.1109/tdsc.2004.2.

[16] B. Burger, P. M. Maffettone, V. V. Gusev, C. M. Aitchison, Y. Bai, X. Wang, X. Li, B. M. Alston, B. Li, R. Clowes, N. Rankin, B. Harris, R. S. Sprick, A. I. Cooper. A mobile robotic chemist. *Nature*, 583(7815):237–241, 2020. DOI: 10.1038/s41586-020-2442-2.

[17] N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, T. He, D. Milsted, M. J. McDermott, M. Gallant, E. D. Cubuk, A. Merchant, H. Kim, A. Jain, C. J. Bartel, K. Persson, Y. Zeng, G. Ceder. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 624(7990):86–91, 2023. DOI: 10.1038/s41586-023-06734-w. See also the Author Correction, *Nature*, 650(8100):E1, 2026, DOI: 10.1038/s41586-025-09992-y.

[18] L. M. Roch, F. Häse, C. Kreisbeck, T. Tamayo-Mendoza, L. P. E. Yunker, J. E. Hein, A. Aspuru-Guzik. ChemOS: Orchestrating autonomous experimentation. *Science Robotics*, 3(19):eaat5559, 2018. DOI: 10.1126/scirobotics.aat5559.

[19] M. Sim et al. ChemOS 2.0: An orchestration architecture for chemical self-driving laboratories. *Matter*, 7(9):2959–2977, 2024. DOI: 10.1016/j.matt.2024.04.022.

[20] B. P. MacLeod et al. Self-driving laboratory for accelerated discovery of thin-film materials. *Science Advances*, 6(20):eaaz8867, 2020. DOI: 10.1126/sciadv.aaz8867.

[21] B. P. MacLeod, F. G. L. Parlane, C. P. Berlinguette. How to build an effective self-driving laboratory. *MRS Bulletin*, 48(2):173–178, 2023. DOI: 10.1557/s43577-023-00476-w.

[22] M. Abolhasani, E. Kumacheva. The rise of self-driving labs in chemical and materials sciences. *Nature Synthesis*, 2(6):483–492, 2023. DOI: 10.1038/s44160-022-00231-0.

[23] G. Tom et al. Self-Driving Laboratories for Chemistry and Materials Science. *Chemical Reviews*, 124(16):9633–9732, 2024. DOI: 10.1021/acs.chemrev.4c00055.

[24] W. Gottstein, L. Bromig, M. Arnold, S. Barazandeh, J. S. Brown, K. A. Brown, et al. Talking to instruments: device-integration standards for self-driving laboratories. ChemRxiv preprint, 6 October 2026. DOI: 10.26434/chemrxiv.15009990.

[25] K. L. Snapp, K. A. Brown. Driving school for self-driving labs. *Digital Discovery*, 2(5):1620–1629, 2023. DOI: 10.1039/d3dd00150d.

[26] D. A. Boiko, R. MacKnight, B. Kline, G. Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. DOI: 10.1038/s41586-023-06792-0.

[27] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, P. Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. DOI: 10.1038/s42256-024-00832-8.

[28] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, D. Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292, 2024.

[29] J. Wei, Y. Yang, X. Zhang, Y. Chen, et al. From AI for Science to Agentic Science: A Survey on Autonomous Scientific Discovery. arXiv:2508.14111, 2025.

[30] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.

[31] R. P. Wierenga, S. M. Golas, W. Ho, C. W. Coley, K. M. Esvelt. PyLabRobot: An open-source, hardware-agnostic interface for liquid-handling robots and accessories. *Device*, 1(4):100111, 2023. DOI: 10.1016/j.device.2023.100111.

[32] SiLA Consortium. *SiLA 2 Standard*. https://sila-standard.org

[33] IEC 62541. *OPC Unified Architecture*. International Electrotechnical Commission.

[34] R. D. Lewis, T. S. Ginsburg, D. Ozgulbas, C. Stone, A. Stroka, A. Cleary, I. T. Foster, N. Paulson. MADSci: A modular Python-based framework to enable autonomous science. *Journal of Open Source Software*, 11(119):9416, 2026. DOI: 10.21105/joss.09416.

[35] R. Vescovi, T. Ginsburg, K. Hippe, D. Ozgulbas, C. Stone, A. Stroka, et al. Towards a modular architecture for science factories. *Digital Discovery*, 2(6):1980–1998, 2023. DOI: 10.1039/d3dd00142c.

[36] B. Bartley, J. Beal, M. Rogers, D. Bryce, R. P. Goldman, B. Keller, P. Lee, V. Biggers, J. Nowak, M. Weston. Building an Open Representation for Biological Protocols. *ACM Journal on Emerging Technologies in Computing Systems*, 19(3):1–21, 2023. DOI: 10.1145/3604568.

[37] A. B. Yoo, M. A. Jette, M. Grondona. SLURM: Simple Linux Utility for Resource Management. In *Job Scheduling Strategies for Parallel Processing* (JSSPP 2003), Lecture Notes in Computer Science 2862, pp. 44–60. Springer, 2003. DOI: 10.1007/10968987_3.

[38] P. Di Tommaso, M. Chatzou, E. W. Floden, P. Prieto Barja, E. Palumbo, C. Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. DOI: 10.1038/nbt.3820.

[39] F. Mölder, K. P. Jablonski, B. Letcher, M. B. Hall, C. H. Tomkins-Tinch, V. Sochat, et al. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021. DOI: 10.12688/f1000research.29032.2.

[40] Google DeepMind. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. 2025. **[TODO: verify citation form and date before submission]**

[41] IEC 61508:2010 (Edition 2.0, parts 1–7). *Functional safety of electrical/electronic/programmable electronic safety-related systems*. International Electrotechnical Commission, 2010.

[42] ISO 10218-1:2025 and ISO 10218-2:2025. *Robotics — Safety requirements — Part 1: Industrial robots; Part 2: Industrial robot applications and robot cells*. International Organization for Standardization, 2025. These third- and second-edition documents replaced the 2011 editions, whose title series was *Robots and robotic devices*.

[43] ISO/TS 15066:2016. *Robots and robotic devices — Collaborative robots*. International Organization for Standardization, 2016. Technical Specification; a revision (ISO/AWI 15066-1) is in development.

[44] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. 2010. https://www.jsonrpc.org/specification

[45] S. Robertson, H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. **[TODO: not independently re-verified; check before submission]**